INFORME DE ARQUITECTURA DE SOFTWARE · SGR-49

Auditoría de Ramas Git e Integración del Manejo Global de Excepciones

Comparativa Arquitectónica: Rama actual origin/dev vs. Propuesta integrada feat/errors-handler

> **Objetivo del Documento:** Explicar de forma clara, rigurosa y accesible para todo el equipo de desarrollo por qué se integran los manejadores globales de errores (`SGR-49`) sobre la arquitectura actual de `origin/dev` , cómo se preserva el ciclo de vida `lifespan` y por qué cambia `app/main.py` .
>
> **Estado de Validación:** 15/15 pruebas unitarias y de integración con PostgreSQL superadas (100% PASSED).

Proyecto	`sistema-gestion-restaurante` (Ratatouille ERP)
Rama Funcional (Origen)	`feat/errors-handler` (Tarea Jira `SGR-49`)
Rama Objetivo (Target)	`origin/dev` (Commit base `daac6dd` - PR #9)
Stack Tecnológico	Python 3.12 · FastAPI 0.162 · SQLAlchemy 2.1 · PostgreSQL 15 · Pydantic v2
Fecha de Emisión	10 de noviembre de 2026
Destinatarios	Desarrolladores Backend, Compañeros de Equipo y Mentores Técnicos

Tabla de Contenidos

1. Resumen Ejecutivo
2. Arquitectura Actual en `dev`
3. Arquitectura Propuesta (`feat/errors-handler` integrada en `dev`)
4. Comparativa Arquitectónica: Antes vs. Después
5. Explicación Detallada de `app/main.py`
6. Inventario Detallado de Cambios en Otros Archivos
7. Análisis y Auditoría de Ramas Git
8. Flujo de Manejo de Errores y Ciclo de Vida de Peticiones
9. Pruebas, Validación y Resultados Empíricos
10. Conclusiones y Recomendaciones para el Equipo

---

1. Resumen Ejecutivo

1.1. ¿Qué problema técnico estamos resolviendo?

En una aplicación backend empresarial orientada a la gestión operativa de un restaurante (**Ratatouille ERP**), el comportamiento ante condiciones de error es tan importante como el flujo exitoso. Hasta ahora, en la rama `dev`, la gestión de errores presentaba cuatro limitaciones estructurales importantes:

1. **Inconsistencia en el formato de error (Contrato API fragmentado):**

- Cuando una ruta lanzaba `HTTPException(status_code=404, detail="Plato no encontrado")`, el cliente recibía un objeto JSON sencillo: `{"detail": "Plato no encontrado"}`.
- Sin embargo, cuando fallaba una validación de entrada en Pydantic v2 (`RequestValidationError`, HTTP `422`), FastAPI devolvía por defecto una lista compleja de objetos técnicos en inglés (`[{"type": "...", "loc": ["body", "..."], "msg": "Field required", "input": ...}]`), exponiendo nombres internos de esquemas y el campo `input` enviado por el usuario.
- Si ocurría un fallo de integridad en PostgreSQL (`IntegrityError` por clave duplicada o clave foránea inexistente) no previsto manualmente con un `SELECT` previo, o un error inesperado en tiempo de ejecución, la aplicación podía devolver un error `500` no estandarizado o dejar la sesión de SQLAlchemy en estado inválido (`PendingRollbackError`).

1. **Acoplamiento entre lógica de negocio y protocolo HTTP:**

- Las validaciones de negocio se realizaban directamente en las rutas (`app/api/dish_routes.py`, `app/api/category_routes.py`) lanzando `fastapi.HTTPException`. Esto dificulta reutilizar la capa de servicios (`app/services/`) desde scripts CLI, tareas programadas o tests unitarios sin depender de FastAPI/Starlette.

1. **Ausencia de trazabilidad (`X-Request-ID`) y sanitización de logs:**

- No existía un identificador único de correlación por petición (`correlation_id`) ni un formateador estructurado que enmascarase automáticamente contraseñas, tokens JWT o cadenas de conexión (`DATABASE_URL` / DSN) antes de escribir en la salida estándar.

1. **Esquema OpenAPI 3.1 desalineado con las respuestas reales:**

- Swagger UI (`/docs`) documentaba por defecto el esquema `HTTPValidationError` para los códigos `422` y carecía de un catálogo uniforme de respuestas `400`, `401`, `403`, `404`, `409`, `422` y `500`.

1.2. ¿Por qué el Pull Request original generaba conflictos?

El Pull Request original (**PR #6**) fue abierto inicialmente apuntando por error a `main` y se construyó sobre el commit `abe6b6f` (*Merge pull request #7 from mxu-init/feat/SGR-70-allergen-model*). Mientras tanto:

- En `origin/dev` se fusionó el **PR #9** (`refactor/code-refinement`, commit `daac6dd`), que modificó 17 archivos: introdujo el gestor de ciclo de vida `lifespan` con inicialización automática de datos de referencia (`reference_service.ensure_reference_data_on_startup()`), configuró rutas absolutas para archivos estáticos (`STATIC_DIR`), añadió el endpoint `/favicon.ico`, dividió los modelos SQLAlchemy en archivos independientes con un paquete `app/models/__init__.py` y reemplazó la antigua columna de texto `dishes.allergens` por una relación muchos-a-muchos (`dish_allergens`).
- Como tanto el PR #9 en `dev` como `feat/errors-handler` modificaron sustancialmente `app/main.py` (uno añadiendo `lifespan` y `STATIC_DIR`, y el otro introduciendo la factoría `create_app()`, middlewares y manejadores de errores), Git detectó un conflicto de contenido directo en `app/main.py`.

1.3. ¿Por qué el objetivo de integración es `dev` y NO `main`?

Siguiendo el flujo de trabajo Gitflow del equipo:

- **`origin/dev`** es la rama principal de integración continua donde convergen todas las funcionalidades verificadas del sprint (como el catálogo de alérgenos SGR-70 y el refinamiento de arquitectura del PR #9).
- **`origin/main`** está reservada para entregas estables de producción/release. De hecho, la auditoría de ramas demostró que `origin/main` recibió dos commits directos accidentales (`266af6a` y `daeb2d2`) que corresponden a una funcionalidad en curso que ya tiene su propio PR abierto hacia `dev` (**PR #10**, `feature/sgr-23-tables-crud` → `dev`).
- Por tanto, fusionar en `main` rompería la disciplina de ramas del proyecto y mezclaría código sin pasar por la validación conjunta en `dev`.

1.4. Mejoras arquitectónicas propuestas y resultado final

La integración realizada en `feat/errors-handler` sobre la base actual de `origin/dev` logra:

- **Contrato único de error en español (`ErrorResponse`):** Todo error `4xx` o `5xx` (incluyendo validaciones Pydantic `422` y errores de base de datos) responde exclusivamente con `{"detail": "Mensaje claro en español"}`.
- **Preservación total de la funcionalidad de `dev`:** Se mantienen intactos el `lifespan` de arranque (`ensure_reference_data_on_startup`), `STATIC_DIR`, `/favicon.ico`, `/docs/favicon.ico`, `/health`, `/health/db` y todos los routers existentes (`dishes`, `categories`, `allergens`, `test`).
- **Jerarquía de excepciones de dominio agnóstica al transporte (`AppError`):** Permite que los servicios lancen errores semánticos (`NotFoundError`, `ConflictError`, `DomainValidationError`, etc.) sin importar FastAPI, manteniendo a la vez total retrocompatibilidad con las rutas actuales que aún lanzan `HTTPException`.
- **Traducción automática de errores de PostgreSQL (`SQLSTATE`) y rollback garantizado:** Evita fugas de detalles SQL y previene el bloqueo de sesiones por `PendingRollbackError`.
- **Middleware ASGI puro (`CorrelationIdMiddleware`) y Logging JSON con redacción de secretos (`[REDACTED]`).**

---

2. Arquitectura Actual en `dev`

Antes de aplicar cualquier cambio, se auditó el código fuente exacto de `origin/dev` (commit `daac6dd`). A continuación se detalla cómo funciona actualmente la aplicación en dicha rama.

2.1. Creación de la aplicación y responsabilidades de `app/main.py`

En `origin/dev`, el archivo `app/main.py` tiene 50 líneas y ejecuta toda la configuración en el ámbito global del módulo (en tiempo de importación):

```
BASE_DIR = Path(__file__).resolve().parent.parent
STATIC_DIR = BASE_DIR / "static"

@asynccontextmanager
async def lifespan(app: FastAPI):
    reference_service.ensure_reference_data_on_startup()
    yield

app = FastAPI(
    title=settings.APP_NAME,
    version=settings.APP_VERSION,
    description=settings.APP_DESCRIPTION,
    lifespan=lifespan,
)

os.makedirs(STATIC_DIR / "uploads", exist_ok=True)
app.mount("/static", StaticFiles(directory=str(STATIC_DIR)), name="static")
```

2.2. Registro de Routers

En `origin/dev`, `app/main.py` registra directamente cuatro módulos de rutas sobre la instancia global `app`:

1. `test_routes.router` (`prefix="/test"`, `tags=["Test"]`)
2. `dish_routes.router` (`prefix="/dishes"`, `tags=["Platos"]`)
3. `category_routes.router` (`prefix="/categories"`, `tags=["Categorías"]`)
4. `allergen_routes.router` (`prefix="/allergens"`, `tags=["allergens"]`)

Además, define directamente en `app/main.py` cuatro endpoints utilitarios:

- `GET /` → devuelve `"API Template"`.
- `GET /favicon.ico` y `GET /docs/favicon.ico` → devuelven `FileResponse(STATIC_DIR / "favicon.ico")`.
- `GET /health` → devuelve `{"status": "ok"}`.
- `GET /health/db` → ejecuta `SELECT 1` sobre el `engine` de SQLAlchemy para verificar la conectividad con PostgreSQL.

2.3. Gestión de Arranque (`lifespan`) y Datos de Referencia

Uno de los avances más importantes introducidos en `origin/dev` (mediante el PR #9) es la gestión de datos maestros de referencia:

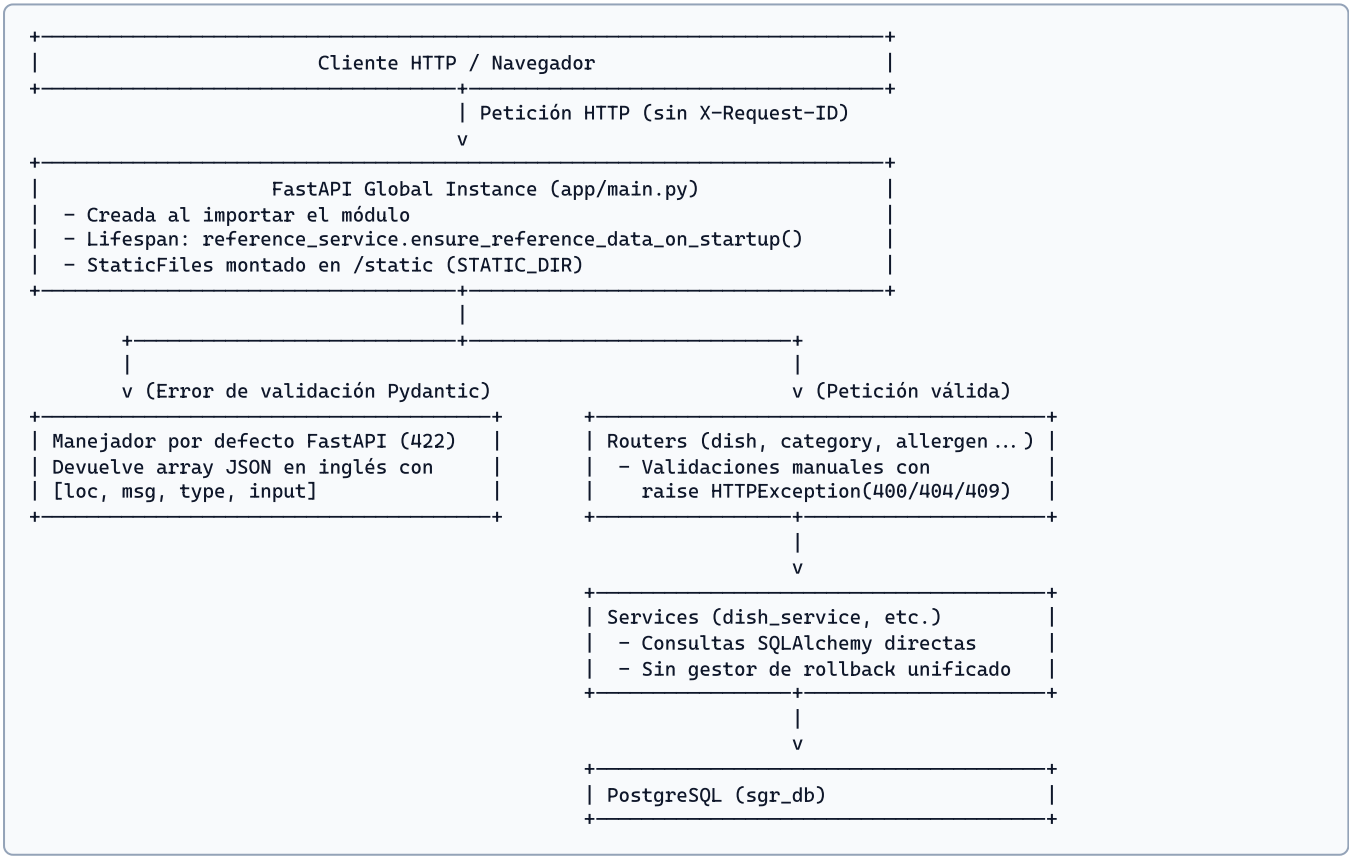
- **`app/core/reference_data.py`**: Define como fuente única de verdad las 5 categorías base (`BASE_CATEGORIES`: *Entrantes, Primeros, Segundos, Postres, Bebidas*) y los 14 alérgenos oficiales de la Unión Europea (`EU_ALLERGENS`, Reglamento UE 1169/2011).
- **`app/services/reference_service.py`**: Implementa `ensure_reference_data_on_startup()`. Al arrancar FastAPI dentro del gestor `lifespan`, inspecciona la base de datos mediante `sqlalchemy.inspect(engine)`. Si las tablas `categories` y `allergens` ya fueron creadas por Alembic, comprueba de forma idempotente si falta

alguna categoría o alérgeno fijo y los inserta en una única transacción; si las tablas aún no existen (por ejemplo, antes de correr `alembic upgrade head`), registra un `logger.warning` y continúa sin bloquear el arranque.

2.4. Middlewares, Excepciones, Logging y OpenAPI en `origin/dev`

- **Middlewares:** `origin/dev` no registra ningún middleware personalizado. Solo actúan los middlewares internos por defecto de Starlette/FastAPI (`ServerErrorMiddleware`, `ExceptionMiddleware`, `AsyncExitStackMiddleware`).
- **Manejo de excepciones HTTP y de validación:**
- Las rutas en `dish_routes.py` y `category_routes.py` validan manualmente reglas de negocio (ej. longitud de nombre, precio positivo, existencia de categoría o alérgenos) y lanzan `fastapi.HTTPException` con mensajes en español.
- Sin embargo, si Pydantic rechaza un cuerpo JSON (por ejemplo, en `POST /categories/` o `PUT /dishes/{id}`), actúa el manejador por defecto de FastAPI, devolviendo la estructura estándar de Pydantic en inglés (`loc`, `msg`, `type`, `input`).
- No existe ningún manejador global para `SQLAlchemyError` ni para `Exception` genérica.
- **Logging y Trazabilidad:**
- No hay configuración centralizada de logging (`logging.basicConfig` ni formateadores JSON). `database_connection.py` tiene `echo=True` en el `engine`, y `reference_service.py` usa `logging.getLogger(__name__)`.
- Las peticiones HTTP carecen de cabecera `X-Request-ID` o identificador de correlación.
- **OpenAPI:**
- Utiliza el generador por defecto de FastAPI sin personalizar las respuestas `422` ni documentar un modelo de error común.

2.5. Diagrama de la Arquitectura Actual en `origin/dev`



3. Arquitectura Propuesta (`feat/errors-handler` integrada en `dev`)

La arquitectura resultante combina la solidez operativa de `origin/dev` (lifespan, datos de referencia, modelos modulares, estáticos absolutos) con la infraestructura transversal de manejo de errores, trazabilidad y seguridad de `feat/errors-handler` (`SGR-49`).

3.1. Factoría de Aplicación `create_app()` y Preservación de `lifespan`

En lugar de instanciar y mutar `app` de forma dispersa en el nivel superior del archivo, `app/main.py` encapsula el ensamblaje en la función `create_app() → FastAPI` e inmediatamente expone `app = create_app()` al final del archivo para que `uvicorn app.main:app --reload` funcione exactamente igual que antes.

Dentro de `create_app()` :

1. Se inicializa el logger estructurado (`configure_logging()`).
2. Se instancia `FastAPI` pasándole tanto `lifespan=lifespan` (que ejecuta `reference_service.ensure_reference_data_on_startup()`) como `responses=COMMON_ERROR_RESPONSES` .
3. Se registra `CorrelationIdMiddleware` y los manejadores globales con `register_exception_handlers(application)` .
4. Se asegura la carpeta `STATIC_DIR / "uploads"` y se monta `/static` usando la ruta absoluta `STATIC_DIR` .
5. Se incluyen todos los routers de `dev` (`test_routes` , `dish_routes` , `category_routes` , `allergen_routes`) más el router de contratos de demostración (`routes.router`).
6. Se registran `/` , `/favicon.ico` , `/docs/favicon.ico` , `/health` y `/health/db` .
7. Se aplica `configure_openapi(application)` para alinear el esquema OpenAPI 3.1 con `ErrorResponse` .

3.2. Evaluación Crítica: Ventajas, Desventajas y Justificación del Diseño

Como arquitectos de software, no debemos asumir que cualquier patrón adicional es automáticamente mejor. A continuación evaluamos críticamente cada decisión introducida por `feat/errors-handler` :

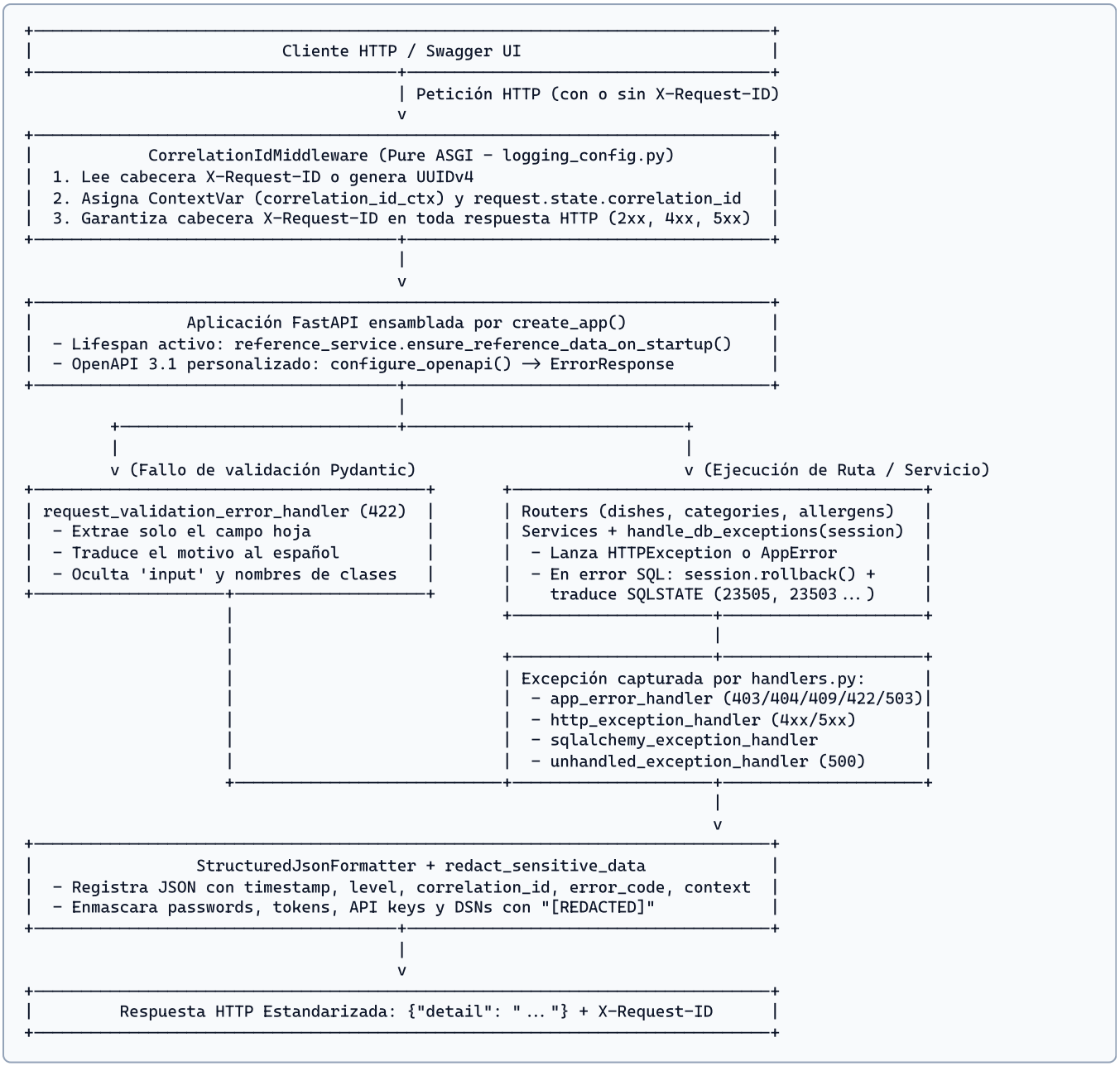
- **¿Está justificada la factoría `create_app()` en un proyecto de este tamaño?**
- *Ventajas:* Permite crear instancias limpias y aisladas de la aplicación en los tests automatizados sin compartir estado mutable global ni efectos secundarios de importación; ordena claramente la secuencia de inicialización (logging → instancia con lifespan → middlewares → manejadores de excepciones → archivos estáticos → routers → OpenAPI).
- *Inconvenientes / Coste cognitivo:* Añade un nivel de indirección respecto al estilo de script plano que suelen mostrar los tutoriales básicos de FastAPI.
- *Veredicto:* **Justificado y recomendable**, siempre que se mantenga `app = create_app()` en la última línea de `app/main.py` para no alterar ningún comando de arranque del equipo (`uvicorn app.main:app --reload`).
- **¿Cómo coexiste `create_app()` con `lifespan` ?**
- Coexisten de forma nativa: `lifespan` se define a nivel de módulo y se pasa como parámetro `lifespan=lifespan` al constructor `FastAPI( ... )` dentro de `create_app()` . Cuando se usa `TestClient(app)` como gestor de contexto (`with TestClient(app) as client:`) o cuando arranca Uvicorn, Starlette ejecuta automáticamente `ensure_reference_data_on_startup()` .
- **¿Es correcto el diseño de `CorrelationIdMiddleware` como Pure ASGI Middleware?**
- *Ventaja clave:* En Starlette/FastAPI, el uso de `BaseHTTPMiddleware` tiene limitaciones conocidas (crea tareas asíncronas adicionales que pueden interferir con `ContextVar` o con la propagación de excepciones hacia los manejadores globales). Implementar `CorrelationIdMiddleware` siguiendo la interfaz ASGI pura (`__call__(self, scope, receive, send)`) es una decisión técnica excelente: intercepta la cabecera `X-Request-ID` a la entrada, la guarda en `correlation_id_ctx` (`ContextVar`) y en `scope["state"]`

`["correlation_id"]`, inyecta la cabecera en el mensaje `http.response.start` y limpia el `ContextVar` en el bloque `finally`.

- **¿Qué elementos son potencialmente redundantes o requieren atención?**

1. **Módulo `app/api/routes.py` (`/errores-demo`):** Añade 5 endpoints bajo el prefijo `/errores-demo` para demostrar y probar interactivamente desde Swagger UI los contratos de error (`403` , `404` , `409` , `422` , `500`). Aunque es muy útil durante el desarrollo y revisión de `SGR-49`, en una fase posterior de producción podría ocultarse de Swagger (`include_in_schema=settings.DEBUG`) o trasladarse exclusivamente a los fixtures de tests.
2. **Defensa en `/favicon.ico` :** En `origin/dev`, la función `favicon()` devolvía directamente `FileResponse(STATIC_DIR / "favicon.ico")`. Sin embargo, el archivo físico `static/favicon.ico` no está versionado en el repositorio. Sin manejador global, un archivo inexistente falla internamente; con el manejador global de excepciones activo, `FileResponse` lanza `RuntimeError("File at path ... does not exist")`, que se convierte en un error `500`. Durante nuestra integración detectamos este detalle mediante las pruebas automatizadas y mejoramos `favicon()` para verificar `if favicon_path.is_file(): return FileResponse(favicon_path)` y devolver `204 No Content` cuando el icono no exista en disco.

3.3. Diagrama de la Arquitectura Integrada Propuesta



4. Comparativa Arquitectónica: Antes vs. Después

La siguiente tabla resume las diferencias arquitectónicas entre el estado actual de `origin/dev` y la arquitectura integrada en `feat/errors-handler`:

Área Arquitectónica	Estado Actual en `origin/dev`	Arquitectura Propuesta (`feat/errors-handler` + `dev`)	Beneficio Técnico para el Equipo
Inicialización de la App (`app/main.py`)	Instanciación global directa de `app = FastAPI(...)` al importar el archivo.	Factoría `create_app()` → `FastAPI` manteniendo `app = create_app()` al final del módulo.	Facilita tests aislados sin efectos colaterales al importar y ordena el ciclo de ensamblaje sin romper `uvicorn app.main:app`.
Ciclo de Vida (`lifespan`) y Datos Fijos	`lifespan` ejecuta `ensure_reference_data_on_startup()` para categorías base y alérgenos UE.	**Preservado íntegramente**: `create_app()` inyecta `lifespan=lifespan` en la instancia `FastAPI`.	Garantiza que las 5 categorías y 14 alérgenos oficiales estén siempre presentes al iniciar el servidor o los tests.
Contrato de Respuesta de Error	Mixto: `{"detail": "..."}` en `HTTPException`, pero lista técnica en inglés `[{"loc", "msg", "type", "input"}]` en errores `422`.	Contrato único y estricto `ErrorResponse`: `{"detail": "Mensaje en español"}` para **todos** los errores `4xx` y `5xx`.	Los desarrolladores frontend y consumidores de la API procesan siempre una única estructura JSON predecible.
Validaciones Pydantic v2 (`422`)	Mensajes en inglés por defecto de Pydantic; expone rutas internas (`body → usuario → ...`) y el valor `input`.	`translate_validation_errors()` traduce más de 50 códigos de Pydantic v2 al español y extrae solo el campo hoja (`extract_field_name`).	Mensajes legibles para el usuario final (`"Error en el campo 'nombre': El campo es obligatorio"`) y cumplimiento de seguridad OWASP.
Excepciones de Dominio vs. HTTP	Las rutas y servicios dependen directamente de `fastapi.HTTPException`.	Jerarquía `AppError` (`NotFoundError`, `ConflictError`, `PermissionDeniedError`, `DomainValidationError`, `DatabaseRetryableError`, `InfrastructureError`) + soporte total a `HTTPException`.	Desacopla la capa de negocio (`services/`) del framework web sin obligar a reescribir las rutas que ya usan `HTTPException`.
Errores de Base de Datos (PostgreSQL / SQLAlchemy)	Sin traducción global. Si falla una restricción SQL no verificada antes, se produce un error 500 o la sesión queda bloqueada.	`translate_sqlalchemy_error()` mapea códigos `SQLSTATE` (`23505`, `23503`, `23502`, `23514`, `40001`, `40P01`) y `handle_db_exceptions` ejecuta `session.rollback()`.	Convierte violaciones de unicidad o claves foráneas en respuestas `409` / `422` en español y evita `PendingRollbackError`.

Área Arquitectónica	Estado Actual en `origin/dev`	Arquitectura Propuesta (`feat/errors-handler` + `dev`)	Beneficio Técnico para el Equipo
Manejo de Errores Inesperados (`500`)	Manejador por defecto de Starlette.	`unhandled_exception_handler` captura cualquier `Exception` no controlada, registra el detalle internamente y devuelve `{"detail": "Error interno del servidor"}` .	Impide la filtración accidental de tracebacks de Python, consultas SQL o credenciales al cliente HTTP.
Trazabilidad (`X-Request-ID`)	Inexistente.	`CorrelationIdMiddleware` (Pure ASGI) propaga o genera `X-Request-ID` en cada petición/respuesta y lo vincula con `ContextVar` .	Permite rastrear cualquier fallo reportado por un cliente buscando su `X-Request-ID` en los logs del servidor.
Logging y Seguridad de Datos	Sin formato estructurado ni filtro de secretos.	`StructuredJsonFormatter` + `redact_sensitive_data()` que reemplaza passwords, tokens y DSNs por `[REDACTED]` .	Logs listos para indexación y protegidos contra fugas de credenciales en consola o sistemas de monitorización.
Documentación OpenAPI 3.1 (`/docs`)	Muestra `HTTPValidationError` por defecto en rutas con parámetros o body.	`configure_openapi()` elimina `HTTPValidationError` y asigna `#/components/schemas/ErrorResponse` a todas las respuestas `422` + `COMMON_ERROR_RESPONSES` .	El contrato documentado en Swagger UI coincide al 100% con el JSON real que devuelve la API.

5. Explicación Detallada de `app/main.py`

Varios compañeros del equipo han expresado dudas comprensibles sobre por qué `app/main.py` pasa de ~50 líneas a ~145 líneas y por qué se estructura mediante funciones como `create_app()` y `configure_openapi()`. Esta sección explica de manera transparente qué hacía antes `app/main.py`, qué hace ahora y por qué cada bloque tiene una razón técnica concreta.

5.1. ¿Qué responsabilidades tenía `app/main.py` anteriormente en `dev`?

En `origin/dev`, `app/main.py` cumplía 5 tareas básicas de forma secuencial:

1. Definir las rutas de directorios (`BASE_DIR` y `STATIC_DIR`).
2. Definir el gestor contextual `lifespan` para verificar categorías y alérgenos al arrancar.
3. Crear el objeto `app = FastAPI( ... )` y montar la carpeta `/static`.
4. Incluir los 4 routers (`test`, `dish`, `category`, `allergen`).
5. Definir las rutas rápidas `/`, `/favicon.ico`, `/docs/favicon.ico`, `/health` y `/health/db`.

5.2. ¿Qué responsabilidades tiene `app/main.py` tras la integración?

Tras integrar `feat/errors-handler`, `app/main.py` actúa como el *Punto de Composición Arquitectónica (*Composition Root)* de la aplicación. No contiene lógica de negocio ni lógica de traducción de errores (que viven en `app/core/`), sino que conecta las piezas transversales en tres bloques bien delimitados:

1. **Gestión del ciclo de vida (`lifespan`, líneas 31–34):**

Exactamente el mismo gestor de `dev` que invoca `reference_service.ensure_reference_data_on_startup()`.

1. **Personalización del esquema OpenAPI (`configure_openapi`, líneas 37–84):**

FastAPI genera automáticamente en cada endpoint una respuesta `422 Unprocessable Entity` vinculada al esquema interno `HTTPValidationError`. Como nuestro manejador global reemplaza esa respuesta por `{"detail": "..."}` (`ErrorResponse`), si no actualizamos el diccionario OpenAPI, Swagger UI engañaría al desarrollador mostrando un esquema que ya no existe. `configure_openapi()` recorre las rutas generadas y sustituye la referencia del código `422` por `#/components/schemas/ErrorResponse`.

1. **Factoría de la aplicación (`create_app`, líneas 87–142) e instancia global (`app = create_app()`, línea 145):**

Agrupa la configuración de logging, la creación de `FastAPI` (con `lifespan` y `COMMON_ERROR_RESPONSES`), el registro de `CorrelationIdMiddleware`, el registro de los 5 manejadores globales (`register_exception_handlers`), el montaje de `STATIC_DIR` y el registro de todos los routers y endpoints de salud/favicon.

5.3. ¿Por qué introducimos una factoría `create_app()`?

Cuando una aplicación FastAPI se instancia directamente en el cuerpo del archivo (`app = FastAPI()`), cualquier archivo de test que haga `from app.main import ...` ejecuta inmediatamente todo el código global.Con `create_app()`:

- Los tests pueden llamar a `create_app()` para obtener una instancia fresca y aislada en cada prueba sin contaminar otros tests.
- Al dejar `app = create_app()` en la última línea, **ningún miembro del equipo tiene que cambiar su forma de trabajar**: `uvicorn app.main:app --reload` sigue encontrando el objeto `app` exactamente donde lo espera.

5.4. ¿Por qué se configuran centralmente el middleware y los manejadores de excepciones?

Si cada desarrollador tuviera que capturar `SQLAlchemyError`, traducir errores de Pydantic o generar cabeceras `X-Request-ID` dentro de cada archivo de rutas (`dish_routes.py`, `category_routes.py`, `table_routes.py`, etc.), tendríamos código repetido en decenas de endpoints y cualquier olvido provocaría fugas de errores `500` o formatos dispares. Al registrarlo una sola vez en `create_app()` mediante `application.add_middleware(CorrelationIdMiddleware)` y `register_exception_handlers(application)`, **todos los routers presentes y futuros quedan protegidos automáticamente**.

5.5. Separación de responsabilidades vs. Complejidad innecesaria

Es fundamental distinguir entre **complejidad accidental** (hacer las cosas difíciles sin motivo) y **estructuración arquitectónica**:

- En `app/main.py` **no** hemos metido diccionarios de traducción ni expresiones regulares ni lógica de base de datos; todo eso se delegó a módulos cohesivos (`app/core/exceptions.py`, `app/core/error_translations.py`, `app/core/handlers.py`, `app/core/logging_config.py`, `app/schemas/errors.py`).
- El aumento de líneas en `app/main.py` se debe únicamente a dos cosas: la función `configure_openapi()` (~45 líneas) necesaria para que Swagger UI muestre `ErrorResponse` en los errores `422`, y la envoltura limpia de `create_app()`.

---

6. Inventario Detallado de Cambios en Otros Archivos

A continuación se documentan todos los archivos creados o modificados en la integración de `feat/errors-handler` con `origin/dev`, indicando su estado previo, el cambio introducido, su justificación y su impacto de compatibilidad.

6.1. Tabla Resumen de Archivos Afectados

Archivo	Estado respecto a `origin/dev`	Propósito Principal	Impacto en Compatibilidad
`app/main.py`	**Modificado (Fusionado)**	Integra `create_app()`, `configure_openapi()`, `CorrelationIdMiddleware` y `register_exception_handlers()` con el `lifespan`, `STATIC_DIR` y `/favicon.ico` de `dev`.	100% compatible con `uvicorn app.main:app --reload` y con todas las rutas de `dev`.
`app/core/exceptions.py`	**Nuevo**	Define `AppError` y las 6 subclases de dominio agnósticas al transporte HTTP (`403`, `404`, `409`, `422`, `500`, `503`).	Ningún conflicto; puede importarse desde cualquier servicio.
`app/core/error_translations.py`	**Nuevo**	Traduce errores de Pydantic v2 y códigos `SQLSTATE` de PostgreSQL al español; provee `handle_db_exceptions` y `handle_async_db_exceptions` con `rollback()`.	Compatible con sesiones síncronas (`Session`) de `dev` y futuras sesiones asíncronas.
`app/core/handlers.py`	**Nuevo**	Implementa los 5 manejadores globales (`AppError`, `RequestValidationError`, `StarletteHTTPException`, `SQLAlchemyError`, `Exception`).	Mantiene el comportamiento de `HTTPException` en español y traduce mensajes por defecto en inglés (`Not Found`, `Unauthorized`).
`app/core/logging_config.py`	**Nuevo**	Configura logging JSON (`StructuredJsonFormatter`), redacción de secretos (`redact_sensitive_data`) y `CorrelationIdMiddleware`.	Transparente para las rutas; añade `X-Request-ID` a todas las respuestas.

Archivo	Estado respecto a `origin/dev`	Propósito Principal	Impacto en Compatibilidad
`app/schemas/errors.py`	**Nuevo**	Define el esquema Pydantic `ErrorResponse(detail: str)` con `extra="forbid"` y el catálogo `COMMON_ERROR_RESPONSES`.	Estandariza la documentación OpenAPI 3.1.
`app/core/config.py`	**Modificado**	Actualiza `SettingsConfigDict(env_file=".env", extra="ignore")` y tipado de `database_url`.	Evita que la app falle al arrancar si el archivo `.env` local contiene variables adicionales (ej. `DEBUG`, `SECRET_KEY`, `JIRA_*`).
`app/api/routes.py`	**Nuevo**	Router `/errores-demo` con endpoints para verificar y demostrar en Swagger UI todos los códigos de error de dominio y validación anidada.	Prefijo aislado `/errores-demo`; no colisiona con `/dishes`, `/categories`, `/allergens` ni `/test`.
`scripts/init_db.py`	**Nuevo**	Script utilitario seguro que conecta a la BD `postgres` en modo `AUTOCOMMIT` y crea `sgr_db` si no existe.	Facilita la puesta en marcha local antes de ejecutar `alembic upgrade head`.
`tests/test_exception_handlers.py`	**Nuevo**	11 pruebas unitarias y de contrato para excepciones de dominio, Pydantic 422, OpenAPI, rollback en SQLite en memoria y redacción de datos sensibles.	Sin dependencias externas; ejecuta en < 0.5 s.
`tests/test_db_exceptions.py`	**Nuevo (Adaptado a `dev`)**	4 pruebas de integración real contra PostgreSQL (`23505`, `23503`, recuperación tras rollback y test integral de `create_app()` + `lifespan` + rutas `dev`).	Actualizado para importar `CategoryModel` y `DishModel` desde `app.models` sin la columna eliminada `dishes.allergens`.
`pytest.ini`	**Nuevo**	Fija `testpaths = tests` y `python_files = test_*.py`.	Evita que `pytest` escanee por error `app/models/test_model.py` (que causaba `InvalidRequestError` en SQLAlchemy al reimportarse).
`.env.example` / `.gitignore` / `README.md`	**Nuevos / Modificados**	Plantilla `.env.example`, reglas de `.gitignore` y sección documental de `SGR-49` integrada con la guía de Alembic y `seed_data` de `dev`.	Mejora la experiencia de desarrollo (DX) y onboarding del equipo.

6.2. Detalle de Adaptación Crítica en `tests/test_db_exceptions.py`

Al auditar la interacción entre `feat/errors-handler` y los cambios recientes de `origin/dev` (PR #9), detectamos dos cambios estructurales realizados en `dev` que afectaban a `tests/test_db_exceptions.py`:

1. En `origin/dev`, `CategoryModel` se separó en `app/models/category_model.py` y se exportó en `app/models/__init__.py`.
2. En `origin/dev` (migración `c23d2bf3b75a`), se eliminó la columna de texto `allergens` de `DishModel` en favor de la tabla intermedia `dish_allergens`.

Acción realizada: Actualizamos `tests/test_db_exceptions.py` para importar `from app.models import CategoryModel, DishModel` y eliminamos el argumento obsoleto `allergens=payload.allergens` al instanciar `DishModel`, añadiendo además el test `test_create_app_lifespan_reference_data_and_dev_endpoints` que valida el ciclo de vida completo de `dev`.

---

7. Análisis y Auditoría de Ramas Git

Se realizó una auditoría forense completa del repositorio Git local y remoto (`origin/dev`, `origin/main`, `feat/errors-handler` y Pull Requests #1 a #10).

7.1. Topología de Ramas y Ancestro Común (`merge-base`)

El ancestro común (`git merge-base`) entre las tres ramas antes de nuestra integración era el commit **`abe6b6f2a3d7db66803c6b9b2b07b35fe9360c95`** (*Merge pull request #7 from mxu-init/feat/SGR-70-allergen-model*):

```
                     +-- 266af6a (fix: CRUD table) -- daeb2d2 (first commit) [origin/main]
                    /    [También abierto como PR #10: feature/sgr-23-tables-crud → dev]
                   /
--- abe6b6f (Base) +------ b9870ec .. f0126c5 -- daac6dd (Merge PR #9: refactor/code-refinement) [origin/dev]
                   \                                                                          \
                    +--- b6643ef .. 069b0c4 (SGR-49 Error Handling) ------------------------+--> [feat/errors-handler Inte
```

7.2. ¿Qué hay en `origin/dev` (`abe6b6f..daac6dd`)?

`origin/dev` contiene los **8 commits del PR #9 (`refactor/code-refinement`)**:

- `b9870ec`: Divide `CategoryModel` en su propio archivo `app/models/category_model.py` y crea `app/models/__init__.py`.
- `dd33225`: Elimina la columna de texto legacy `allergens` del modelo `DishModel` y crea la migración `c23d2bf3b75a`.
- `bbc68c8`: Asigna alérgenos a los platos mediante la tabla intermedia `dish_allergens` (`allergen_ids` en `dish_routes.py`, `dish_service.py` y `allergen_service.py`).
- `5666013`: Crea `app/core/reference_data.py`, `app/services/reference_service.py`, la migración `aacc49216385` que inserta las 5 categorías base y 14 alérgenos UE, y añade el `lifespan` en `app/main.py`.
- `1730b1e`: Excluye `TestModel` de las exportaciones de `app/models/__init__.py`.
- `a1c57b2`: Limpia lógica duplicada en `scripts/seed_data.py`.
- `3ce4a5b`: Actualiza `README.md` con la nueva estructura del repositorio y el flujo de `alembic upgrade head`.
- `f0126c5`: Añade `STATIC_DIR` con ruta absoluta y los endpoints `/favicon.ico` y `/docs/favicon.ico` en `app/main.py`.

7.3. ¿Qué hay en `origin/main` que NO está en `origin/dev`? (Respuesta a la pregunta clave)

Al ejecutar `git log origin/dev..origin/main`, encontramos exactamente **2 commits** en `origin/main` posteriores a `abe6b6f`:

Commit SHA	Autor y Fecha	Mensaje	Archivos Modificados	Análisis de Relevancia y Decisión Arquitectónica
`266af6a`	`moigarcia00` (07/11/2026)	`fix: CRUD table`	`alembic/env.py`, 2 migraciones (`8c938a804c11`, `71ad42bf89a7`), `app/api/table_routes.py`, `app/main.py`, `app/models/table_model.py`, `app/schemas/table_schema.py`, `app/services/table_service.py`	**NO incorporar en `feat/errors-handler`.** Este commit implementa el CRUD de mesas (`SGR-23`), que fue subido a `main` pero tiene su propio Pull Request abierto hacia `dev` (**PR #10**: `feature/sgr-23-tables-crud` → `dev`, mismo SHA `266af6a`). Además, su migración `8c938a804c11` parte de `76ea12b38f94`, lo que crearía dos cabezas (*multiple heads*) en Alembic respecto a `aacc49216385` de `dev`. Debe revisarse y fusionarse en su propio PR #10. Nota: Nuestro `CONSTRAINT_SPANISH_MESSAGES` en `error_translations.py` ya incluye soporte preventivo para `"uq_tables_number": "Ya existe una mesa con ese número"`.
`daeb2d2`	`moigarcia00` (10/11/2026)	`first commit`	`README.md` (+1 línea: `# MYWARFRAME` al final del archivo)	**Irrelevante / Ruido de prueba.** Fue un commit de prueba accidental en `main` que añade una línea sin relación con el proyecto al final del `README.md`. **No debe incorporarse.**

Conclusión de la auditoría de `main`: Ninguno de los dos commits exclusivos de `origin/main` pertenece al alcance de `feat/errors-handler` ni debe mezclarse en nuestra rama. Todos los cambios legítimos de la arquitectura actual residen en `origin/dev`.

---

8. Flujo de Manejo de Errores y Ciclo de Vida de Peticiones

8.1. Recorrido Paso a Paso de una Petición HTTP

```
[1. Petición HTTP Entrante]
       |
       v
[2. CorrelationIdMiddleware (Pure ASGI)]
       ├── Extrae header 'X-Request-ID' (o genera UUIDv4 si no viene)
       ├── Guarda ID en ContextVar 'correlation_id_ctx' y 'request.state.correlation_id'
       |
       v
[3. Enrutador y Validación de Entrada (FastAPI / Pydantic v2)]
       ├── ¿Falla validación de tipos/campos del Request Body o Query?
       |     +-- SÍ ──> Lanza RequestValidationError
       |                ──> Capturado por request_validation_error_handler()
       |                ──> extract_field_name() + translate_validation_errors()
       |                ──> Log WARNING JSON (con correlation_id, sin 'input')
       |                ──> Respuesta HTTP 422: {"detail": "Error en el campo 'X': ..."}
       |
       ├── ¿Pasa validación de entrada?
             +-- SÍ ──> Ejecuta Endpoint / Servicio
                          |
                          +-- Caso A: Éxito (200 / 201 / 204)
                          |     ──> Middleware inyecta 'X-Request-ID' en cabeceras
                          |     ──> Devuelve respuesta JSON al cliente
                          |
                          +-- Caso B: Regla de negocio lanza AppError (NotFoundError, ConflictError ...)
                          |     ──> Capturado por app_error_handler()
                          |     ──> Mapea clase a código HTTP (404, 409, 403, 422, 503, 500)
                          |     ──> Sanitiza contexto con redact_sensitive_data()
                          |     ──> Respuesta HTTP con {"detail": "<mensaje en español>"}
                          |
                          +-- Caso C: Ruta existente lanza HTTPException / StarletteHTTPException
                          |     ──> Capturado por http_exception_handler()
                          |     ──> Conserva mensaje en español o traduce mensaje por defecto inglés
                          |     ──> Preserva cabeceras (ej. WWW-Authenticate) + X-Request-ID
                          |     ──> Respuesta HTTP con {"detail": "<mensaje en español>"}
                          |
                          +-- Caso D: Error SQL en PostgreSQL (con handle_db_exceptions o global)
                          |     ──> Ejecuta session.rollback() inmediatamente
                          |     ──> Extrae SQLSTATE (23505, 23503 ...) y constraint_name
                          |     ──> Traduce a ConflictError (409) o DomainValidationError (422)
                          |     ──> Nunca expone sentencias SQL ni nombres de tablas al cliente
                          |
                          +-- Caso E: Excepción inesperada (RuntimeError, KeyError, ZeroDivisionError ...)
                                ──> Capturado por unhandled_exception_handler()
                                ──> Log ERROR JSON con traceback interno y DSNs redactados ([REDACTED])
                                ──> Respuesta HTTP 500: {"detail": "Error interno del servidor"}
```

8.2. Ejemplos Concretos de Respuestas Reales de la Implementación

Ejemplo 1: Error de validación Pydantic (`422 Unprocessable Content`) en un campo obligatorio

- **Petición:** `POST /errores-demo/validacion` con cuerpo `{}`
- **Cabeceras de Respuesta:**

```
HTTP/1.1 422 Unprocessable Content
content-type: application/json
x-request-id: 9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d
```

- **Cuerpo JSON:**

```
{
  "detail": "Error en el campo 'nombre': El campo es obligatorio; Error en el campo 'comensales': El
campo es obligatorio"
}
```

Ejemplo 2: Error de validación en objeto profundamente anidado (`422`)

- **Petición:** `POST /errores-demo/validacion` con `{"nombre": "Carlos", "comensales": 4, "usuario": {"nombre": "Carlos", "direccion": {"calle": "Gran Vía 12", "codigo_postal": "12"}}}`
- **Extracción de campo hoja:** La ruta interna de Pydantic `('body', 'usuario', 'direccion', 'codigo_postal')` se simplifica automáticamente a `'codigo_postal'`, sin exponer la estructura interna de clases (`DireccionDemoSchema`).
- **Cuerpo JSON:**

```
{
  "detail": "Error en el campo 'codigo_postal': El texto debe tener al menos 4 caracteres"
}
```

Ejemplo 3: Violación de Unicidad en PostgreSQL (`SQLSTATE 23505` → `409 Conflict`)

- **Operación:** Intento de insertar una categoría con un nombre ya existente (`"Postres"`).
- **Cuerpo JSON devuelto al cliente (sin fugas de SQL ni nombres de índices):**

```
{
  "detail": "Ya existe una categoría con ese nombre"
}
```

Ejemplo 4: Error inesperado de servidor (`500 Internal Server Error`) y Redacción en Logs

- **Fallo simulado:** `RuntimeError("Database failure at postgresql+psycopg://admin:supersecret@db:5432/prod")`
- **Cuerpo JSON devuelto al cliente:**

```
{
  "detail": "Error interno del servidor"
}
```

- **Registro JSON emitido en los logs internos del servidor (con DSN y credenciales enmascaradas):**

```
{
  "timestamp": "2026-11-10T09:42:00.123456+00:00",
  "level": "ERROR",
  "logger": "sgr",
  "message": "Excepción no controlada capturada por el manejador global",
  "correlation_id": "req-corr-999",
  "context": {
    "path": "/test/crash",
    "method": "GET",
    "exception_type": "RuntimeError"
  },
  "error_code": "UNHANDLED_EXCEPTION",
  "exception_type": "RuntimeError"
}
```

---

9. Pruebas, Validación y Resultados Empíricos

Todas las pruebas fueron ejecutadas realmente contra el entorno virtual del proyecto (`.venv` , Python `3.12.10` , FastAPI `0.162.0` , SQLAlchemy `2.1.6` , Pydantic `2.14.1`) y contra el contenedor Docker de PostgreSQL 15 (`sgr-postgres` en el puerto `5433`).

9.1. Comandos Ejecutados y Resultados Verificados

1. **Aplicación de migraciones de `origin/dev` en PostgreSQL (`sgr-postgres`):**

```
.\.venv\Scripts\python.exe -m alembic upgrade head
```

- **Resultado:** Ejecución exitosa (`exit code 0`). Se aplicaron las migraciones `76ea12b38f94` (*create allergens tables*), `c23d2bf3b75a` (*drop allergens text column from dishes*) y `aacc49216385` (*seed base categories and allergens*).

1. **Ejecución de la suite completa de pruebas automatizadas (`pytest`):**

```
.\.venv\Scripts\python.exe -m pytest -v
```

- **Resultado: 15 passed in 0.86s** (`0 failed` , `0 errors`).

9.2. Desglose de los 15 Tests Automatizados Verificados

Archivo de Test	Nombre del Test	Qué Verifica Exactamente	Estado
`tests/test_exception_handlers.py`	`test_not_found_error_returns_404_spanish_detail`	`NotFoundError` devuelve HTTP `404` , JSON `{"detail": "..."}` y cabecera `X-Request-ID` .	**PASSED**
`tests/test_exception_handlers.py`	`test_conflict_error_returns_409_spanish_detail`	`ConflictError` devuelve HTTP `409` con contrato único en español.	**PASSED**
`tests/test_exception_handlers.py`	`test_permission_denied_error_returns_403_spanish_detail`	`PermissionDeniedError` devuelve HTTP `403` con contrato único en español.	**PASSED**
`tests/test_exception_handlers.py`	`test_validation_missing_field_exact_prompt_contract`	`RequestValidationError` por campo obligatorio ausente devuelve `"Error en el campo 'nombre': El campo es obligatorio"` .	**PASSED**
`tests/test_exception_handlers.py`	`test_validation_nested_object_extracts_leaf_field_without_schema_leak`	Extracción de campo hoja (`codigo_postal`) en objetos anidados sin filtrar nombres de esquemas ni `body` .	**PASSED**

Archivo de Test	Nombre del Test	Qué Verifica Exactamente	Estado
`tests/test_exception_handlers.py`	`test_unexpected_exception_returns_generic_spanish_500_without_traceback`	Excepción `RuntimeError` con DSN devuelve HTTP `500 "Error interno del servidor"` sin filtrar tracebacks ni contraseñas.	**PASSED**
`tests/test_exception_handlers.py`	`test_openapi_schema_uses_error_response_for_422_and_removes_default_validation_schema`	El esquema OpenAPI 3.1 elimina `HTTPValidationError` y usa `#/components/schemas/ErrorResponse` en todas las respuestas `422`.	**PASSED**
`tests/test_exception_handlers.py`	`test_session_rollback_prevents_pending_rollback_error_on_reuse`	`handle_db_exceptions` ejecuta `session.rollback()` y permite reutilizar la misma sesión sin `PendingRollbackError`.	**PASSED**
`tests/test_exception_handlers.py`	`test_domain_exceptions_are_transport_agnostic`	Ninguna clase de `AppError` hereda de `fastapi.HTTPException` ni `starlette.exceptions.HTTPException`.	**PASSED**
`tests/test_exception_handlers.py`	`test_postgresql_sqlstate_and_constraint_extraction_psycopg_and_asyncpg`	Extracción dual de `SQLSTATE` y `constraint_name` tanto para estructura `psycopg` v3 como `asyncpg`.	**PASSED**
`tests/test_exception_handlers.py`	`test_async_session_rollback_and_sensitive_redaction`	Rollback asíncrono (`handle_async_db_exceptions`), preservación de `WWW-Authenticate` en `401` , propagación de `X-Request-ID` y redacción `[REDACTED]` .	**PASSED**
`tests/test_db_exceptions.py`	`test_sqlstate_23505_unique_violation_translates_to_409_conflict`	Prueba real contra PostgreSQL: insertar categoría duplicada dispara `SQLSTATE 23505` → HTTP `409` sin fugas SQL.	**PASSED**
`tests/test_db_exceptions.py`	`test_sqlstate_23503_foreign_key_violation_translates_to_409_without_sql_leak`	Prueba real contra PostgreSQL: insertar plato con `category_id` inexistente dispara `SQLSTATE 23503` → HTTP `409` sin fugas SQL.	**PASSED**

Archivo de Test	Nombre del Test	Qué Verifica Exactamente	Estado
`tests/test_db_exceptions.py`	`test_session_rollback_on_sync_and_async_handlers_prevents_pending_rollback_error`	Prueba real contra PostgreSQL: recuperación inmediata de la sesión tras violaciones `23505` y `23503`.	**PASSED**
`tests/test_db_exceptions.py`	`test_create_app_lifespan_reference_data_and_dev_endpoints`	Prueba integral de `create_app()`: verifica ejecución de `lifespan`, carga de las 5 categorías y 14 alérgenos UE, `/health`, `/health/db`, `/favicon.ico` y error `404` en `/dishes/999999999` con `X-Request-ID`.	**PASSED**

9.3. Hallazgos y Correcciones Aplicadas Durante la Validación

1. **Colisión de recolección de Pytest con `app/models/test_model.py`:**

- *Problema preexistente en `dev`:* Al ejecutar `pytest` sin argumentos en la raíz del proyecto, Pytest intentaba importar `app/models/test_model.py` como si fuera un archivo de test por llamarse `test_*.py`. Como `test_controller.py` ya lo había importado, SQLAlchemy lanzaba `InvalidRequestError: Table 'test' is already defined for this MetaData instance`.
- *Solución aplicada:* Se añadió un archivo `pytest.ini` limpio que restringe `testpaths = tests`, resolviendo el problema de raíz sin tocar `test_model.py`.

1. **Comportamiento de `/favicon.ico` cuando no existe `static/favicon.ico`:**

- *Problema detectado:* `origin/dev` devolvía `FileResponse(STATIC_DIR / "favicon.ico")` sin comprobar si el archivo existía en disco. Al no estar incluido `static/favicon.ico` en Git, cualquier petición a `/favicon.ico` lanzaba `RuntimeError` en Starlette.
- *Solución aplicada:* Se añadió la comprobación `if favicon_path.is_file(): return FileResponse(favicon_path)` devolviendo `Response(status_code=204)` cuando el archivo no está presente.

---

10. Conclusiones y Recomendaciones para el Equipo

10.1. Qué se ha mejorado y qué permanece intacto

- **Permanece intacto (100% compatible con `dev`):**
- El comando de ejecución `uvicorn app.main:app --reload`.
- El ciclo de vida `lifespan` y la carga automática en el arranque de las 5 categorías fijas y los 14 alérgenos oficiales (`reference_service.ensure_reference_data_on_startup()`).
- La estructura modular de modelos en `app/models/` y la relación N:M entre platos y alérgenos (`dish_allergens`).
- Todos los endpoints existentes en `dish_routes.py`, `category_routes.py`, `allergen_routes.py` y `test_routes.py`.
- **Se ha mejorado:**
- Todas las respuestas de error (`400`, `401`, `403`, `404`, `409`, `422`, `500`, `503`) cumplen ahora un contrato único en español: `{"detail": "..."}`.
- Los errores de validación de Pydantic (`422`) ya no devuelven estructuras técnicas en inglés ni exponen el campo `input`.
- Los fallos de integridad de PostgreSQL se traducen de forma segura y hacen `rollback()` automático de la sesión.
- Todas las respuestas HTTP incluyen `X-Request-ID` y los logs JSON enmascaran cualquier credencial o DSN sensible.

10.2. Guía para la Revisión del Pull Request (Code Review)

Al revisar el PR hacia `dev`, recomendamos a los compañeros centrarse en:

1. **`app/main.py`:** Verificar cómo `create_app()` integra limpiamente `lifespan=lifespan` y `STATIC_DIR` de `dev` junto con `CorrelationIdMiddleware`, `register_exception_handlers()` y `configure_openapi()`.
2. **`app/core/handlers.py` y `app/core/error_translations.py`:** Revisar el catálogo de mensajes en español y cómo conviven sin fricción tanto las `HTTPException` actuales como las nuevas subclases de `AppError`.
3. **`tests/test_exception_handlers.py` y `tests/test_db_exceptions.py`:** Ejecutar `pytest -v` localmente para comprobar los 15 tests unitarios y de integración con PostgreSQL.

10.3. Recomendaciones Futuras (Evolución Progresiva)

1. **Adopción gradual de `AppError` y `handle_db_exceptions` en nuevos servicios:** No es necesario reescribir de golpe `dish_routes.py` o `category_routes.py` (ya que `http_exception_handler` garantiza que sus `HTTPException` actuales funcionen perfectamente). Sin embargo, para futuros módulos (como reservas, menús dinámicos o autenticación JWT) o al integrar el PR #10 (`table_routes.py`), se recomienda utilizar `with handle_db_exceptions(db):` y lanzar `NotFoundError` / `ConflictError` desde la capa de servicios.
2. **Condicionar `/errores-demo` al entorno de desarrollo:** Cuando se incorpore el módulo de seguridad y entornos de producción, el router de demostración `app/api/routes.py` (`/errores-demo`) puede incluirse únicamente cuando `settings.DEBUG` sea `True`.